Writing functions in R

Monday, June 15, 2026

Today: Monday June 15

Before class

- **Perusall Reading** (Canvas): *R4DS* Ch. 25: Functions
- **HW 4** due tonight, 11:59 p.m.

Plan for today

- *Why* functions: the cost of copy-and-paste
- **Vector functions**: `rescale01()`, mutate helpers, summary helpers
- **Data frame functions** and the embracing operator `{ }`
- **Plot functions** with `ggplot2`
- Naming & style; in-class activity

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 25 · tidyverse style guide

Why write functions?

The rule of three

> *"You should consider writing a function whenever you've copied and pasted a block of code more than twice." - R4DS*

Functions have four big advantages over copy-and-paste:

1. You give the code an **evocative name**, so it's easier to read.
2. As requirements change, you update in **one place** rather than many.

3. You **eliminate copy-paste mistakes** (changing a name in one spot but not another).
4. You can **reuse work** across projects.

Writing functions allows us to go from *using* tidyverse verbs to *writing our own tools*.

Spot the bug

This code rescales every column to run from 0 to 1. Can you find the copy-paste bug?

```
df <- tibble(a = rnorm(5), b = rnorm(5), c = rnorm(5), d = rnorm(5))

df |> mutate(
  a = (a - min(a, na.rm = TRUE)) / (max(a, na.rm = TRUE) - min(a, na.rm = TRUE)),
  b = (b - min(a, na.rm = TRUE)) / (max(b, na.rm = TRUE) - min(b, na.rm = TRUE)),
  c = (c - min(c, na.rm = TRUE)) / (max(c, na.rm = TRUE) - min(c, na.rm = TRUE)),
  d = (d - min(d, na.rm = TRUE)) / (max(d, na.rm = TRUE) - min(d, na.rm = TRUE)),
)
```

> **Warning**
>
> Line `b` divides by `min(a, ...)` instead of `min(b, ...)`. Copy-paste invites this kind of typo, but writing this as a function prevents the possibility of this type of bug altogether.

Vector functions

What varies vs. stays the same

Pull the repeated line out and line the copies up. The pattern jumps out:

```
(a - min(a, na.rm = TRUE)) / (max(a, na.rm = TRUE) - min(a, na.rm = TRUE))
(b - min(b, na.rm = TRUE)) / (max(b, na.rm = TRUE) - min(b, na.rm = TRUE))
(c - min(c, na.rm = TRUE)) / (max(c, na.rm = TRUE) - min(c, na.rm = TRUE))
```

Replace the part that **changes** with :

```
(  - min( , na.rm = TRUE)) / (max( , na.rm = TRUE) - min( , na.rm = TRUE))
```

The only thing that varies is the vector. That becomes our function **argument**.

The anatomy of a function

To write a function you need three things:

- a **name**: here `rescale01` (it rescales a vector to 0–1),
- the **arguments**: the things that vary; here just one, call it `x`,
- the **body**: the code that's repeated.

```
name <- function(arguments) {
  body
}
```

```
rescale01 <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}
```

Always test on simple inputs

Before trusting a function, test run it with inputs where you already know the answer:

```
rescale01(c(-10, 0, 10))
```

```
[1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
[1] 0.00 0.25 0.50   NA 1.00
```

Now the `mutate()` is short, readable, and more bug-resistant:

```
df <- tibble(a = rnorm(5), b = rnorm(5), c = rnorm(5), d = rnorm(5))
df |> mutate(a = rescale01(a), b = rescale01(b),
             c = rescale01(c), d = rescale01(d))
```

```
# A tibble: 5 x 4
      a     b     c     d
  <dbl> <dbl> <dbl> <dbl>
1 1     0.383 0     0
2 0     1     0.922 0.688
3 0.441 0.138 0.624 0.531
4 0.526 0.724 0.171 1
5 0.351 0     1     0.568
```

Changing a function in one place

Because the logic is in *one* spot rather than 4+, improving it is easy. `range()` finds the min and max in a single step:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(c(1:10, Inf))
```

```
 [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667
 [8] 0.7777778 0.8888889 1.0000000       Inf
```

> Tip
>
> `finite = TRUE` tells `range()` to ignore `Inf`. We fixed two behaviors at once.

"Mutate" functions

A **mutate function** returns an output the *same length* as its input, so it slots into `mutate()` and `filter()`.

```
# z-score: center and scale to mean 0, sd 1
z_score <- function(x) {
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
}

# clamp values into [min, max]
clamp <- function(x, min, max) {
  case_when(
    x < min ~ min,
    x > max ~ max,
    .default = x
  )
}
clamp(1:10, min = 3, max = 7)
```

```
 [1] 3 3 3 4 5 6 7 7 7 7
```

Mutate functions aren't just for numbers

```
# capitalize the first letter
first_upper <- function(x) {
  str_sub(x, 1, 1) <- str_to_upper(str_sub(x, 1, 1))
  x
}
first_upper("hello")
```

```
[1] "Hello"
```

```
# strip $, commas, % then convert to a number
clean_number <- function(x) {
  is_pct <- str_detect(x, "%")
  num <- x |>
    str_remove_all("%") |> str_remove_all(",") |> str_remove_all(fixed("$")) |>
    as.numeric()
  if_else(is_pct, num / 100, num)
}
clean_number(c("$12,300", "45%"))
```

```
[1] 12300.00     0.45
```

"Summary" functions

A **summary function** collapses a vector to a *single value*, so it slots into `summarize()`.

```
# coefficient of variation: sd relative to the mean
cv <- function(x, na.rm = FALSE) {
  sd(x, na.rm = na.rm) / mean(x, na.rm = na.rm)
}

# a memorable name for a common pattern
n_missing <- function(x) {
  sum(is.na(x))
}

flights |> summarize(
  n_missing_delay = n_missing(dep_delay),
  cv_delay        = cv(dep_delay, na.rm = TRUE)
)
```

```
# A tibble: 1 x 2
  n_missing_delay cv_delay
            <int>    <dbl>
1            8255     3.18
```

> Tip
>
> Note the **default argument** `na.rm = FALSE`. You can override it, but don't have to. If you don't specify, it won't automatically fail, but rather falls back to the default.

Data frame functions

Repeating verbs, not just expressions

Vector functions pull out code repeated *inside* a verb. But often you repeat the **verbs themselves**. A **data frame function**:

- takes a data frame as its **first argument**,
- takes extra arguments saying what to do,
- returns a data frame (or a vector).

The natural first attempt looks like this, but it fails:

```
grouped_mean <- function(df, group_var, mean_var) {
  df |>
    group_by(group_var) |>
    summarize(mean(mean_var))
}
flights |> grouped_mean(origin, dep_delay)
#> Error: Column `group_var` is not found.
```

The problem: indirection

dplyr uses **tidy evaluation** so you can write `dep_delay` instead of `flights$dep_delay`. That's great most of the time, but inside a function, `group_by(group_var)` looks for a column *named* `group_var` that doesn't exist.

We need to tell dplyr: *don't use the argument's name, look* **inside** *it.*

The solution is **embracing** using `{ }`.

> **Tip**
>
> `{ var }` is analogous to looking **down a tunnel**: it makes dplyr look inside `var` rather than searching for a column called `var`.

Embracing fixes it

```
grouped_mean <- function(df, group_var, mean_var) {
  df |>
    group_by({{ group_var }}) |>
    summarize(mean = mean({{ mean_var }}, na.rm = TRUE), .groups = "drop")
}
flights |> grouped_mean(origin, dep_delay)
```

```
# A tibble: 3 x 2
  origin  mean
  <chr>  <dbl>
1 EWR     15.1
2 JFK     12.1
3 LGA     10.3
```

Now `grouped_mean(origin, dep_delay)` really does group by `origin` and average `dep_delay`.

When do I need to embrace?

Two kinds of tidy evaluation:

Type	Used by	You can…
Data-masking	`filter()`, `arrange()`, `summarize()`, `group_by()`, `mutate()`	*compute* with variables (`x + 1`)
Tidy-selection	`select()`, `relocate()`, `rename()`, `pivot_wider()`	*select* variables (`a:x`, `starts_with()`)

If an argument gets passed to a verb that uses tidy evaluation, typically we want to **embrace it**.

A handy summary helper

Wrap up the summaries you always run during exploration:

```
summary6 <- function(data, var) {
  data |> summarize(
    min = min({{ var }}, na.rm = TRUE),
    mean = mean({{ var }}, na.rm = TRUE),
    median = median({{ var }}, na.rm = TRUE),
    max = max({{ var }}, na.rm = TRUE),
    n = n(),
    n_miss = sum(is.na({{ var }})),
    .groups = "drop"
  )
}
diamonds |> group_by(cut) |> summary6(carat)
```

```
# A tibble: 5 x 7
  cut         min  mean median   max     n n_miss
  <ord>     <dbl> <dbl>  <dbl> <dbl> <int>  <int>
1 Fair       0.22 1.05    1     5.01  1610      0
2 Good       0.23 0.849   0.82  3.01  4906      0
3 Very Good  0.2  0.806   0.71  4    12082      0
4 Premium    0.2  0.892   0.86  4.01 13791      0
5 Ideal      0.2  0.703   0.54  3.5  21551      0
```

Because it wraps `summarize()`, it works on both **grouped** data and on **computed** variables (`summary6(log10(carat))`).

count() with proportions

```
count_prop <- function(df, var, sort = FALSE) {
  df |>
    count({{ var }}, sort = sort) |>
    mutate(prop = n / sum(n))
}
diamonds |> count_prop(clarity)
```

```
# A tibble: 8 x 3
  clarity     n   prop
  <ord>   <int>  <dbl>
1 I1        741 0.0137
2 SI2      9194 0.170
3 SI1     13065 0.242
4 VS2     12258 0.227
5 VS1      8171 0.151
6 VVS2     5066 0.0939
7 VVS1     3655 0.0678
8 IF       1790 0.0332
```

Only `var` is embraced; here, `df` and `sort` are ordinary arguments.

Selecting inside a data-masking function

What if you want to group by *several* user-supplied columns? `group_by()` is data-masking, not tidy-selection, so `c(year, month, day)` breaks. The fix is **`pick()`**:

```
count_missing <- function(df, group_vars, x_var) {
  df |>
    group_by(pick({{ group_vars }})) |>
    summarize(n_miss = sum(is.na({{ x_var }})), .groups = "drop")
}
flights |> count_missing(c(year, month, day), dep_time)
```

```
# A tibble: 365 x 4
    year month   day n_miss
   <int> <int> <int>  <int>
 1  2013     1     1      4
 2  2013     1     2      8
 3  2013     1     3     10
 4  2013     1     4      6
 5  2013     1     5      3
 6  2013     1     6      1
 7  2013     1     7      3
 8  2013     1     8      4
 9  2013     1     9      5
10  2013     1    10      3
# i 355 more rows
```

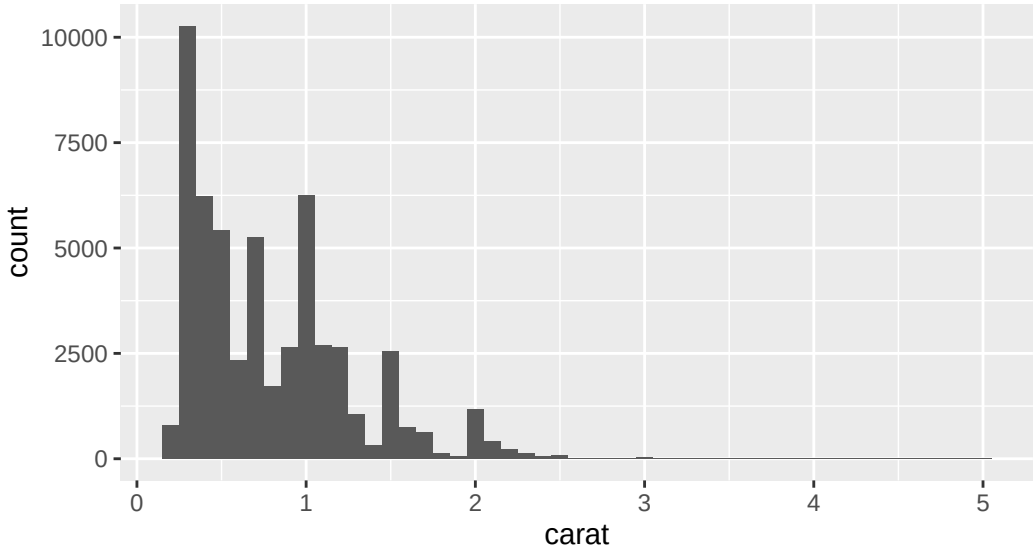
`pick()` lets you use **tidy-selection** inside a **data-masking** function.

Plot functions

Functions that return a plot

`aes()` is a data-masking function, so the *same* `{ }` trick builds plotting functions:

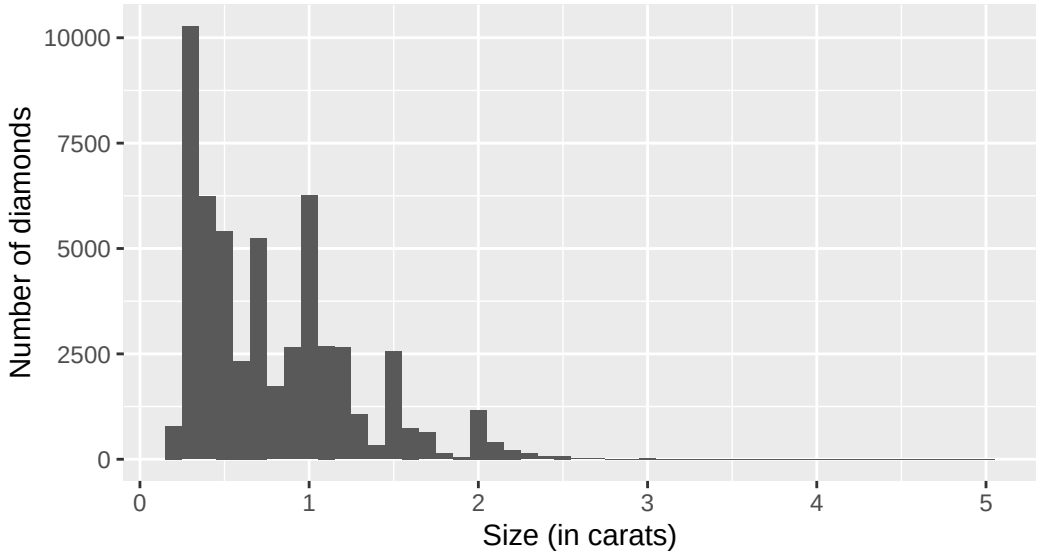
```
histogram <- function(df, var, binwidth = NULL) {
  df |>
    ggplot(aes(x = {{ var }})) +
    geom_histogram(binwidth = binwidth)
}
diamonds |> histogram(carat, 0.1)
```



You can still add layers

A plot function returns a ggplot object, so keep building with `+` (not `|>`):

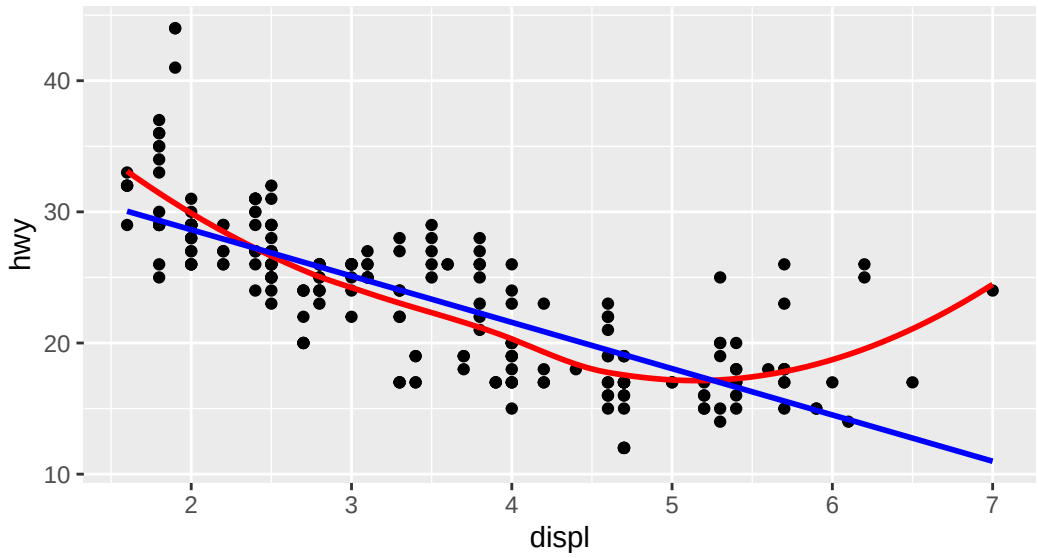
```
diamonds |>
  histogram(carat, 0.1) +
  labs(x = "Size (in carats)", y = "Number of diamonds")
```



More variables

Embrace each variable you pass to `aes()`:

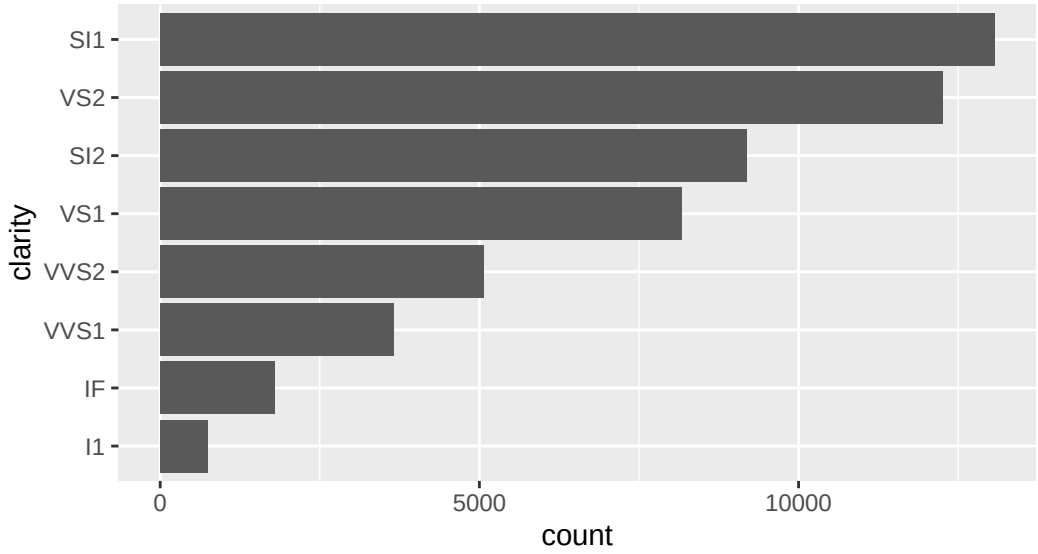
```
linearity_check <- function(df, x, y) {
  df |>
    ggplot(aes(x = {{ x }}, y = {{ y }})) +
    geom_point() +
    geom_smooth(method = "loess", formula = y ~ x, color = "red", se = FALSE) +
    geom_smooth(method = "lm", formula = y ~ x, color = "blue", se = FALSE)
}
mpg |> linearity_check(displ, hwy)
```



The walrus operator :=

To put an embraced name on the **left** of =, use :=:

```
sorted_bars <- function(df, var) {
  df |>
    mutate({{ var }} := fct_rev(fct_infreq({{ var }}))) |>
    ggplot(aes(y = {{ var }})) +
    geom_bar()
}
diamonds |> sorted_bars(clarity)
```

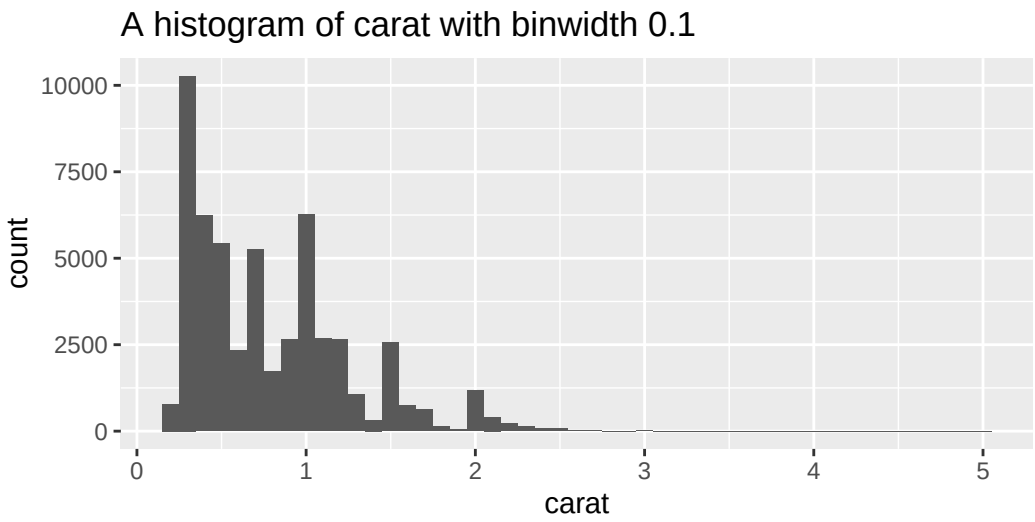


R's syntax won't allow `{ var } =`, so tidy evaluation uses `:=` to mean the same thing.

Labeling with the variable name

`rlang::englue()` works like `str_glue()` but also understands `{ }`, inserting the variable's *name* into a string:

```
histogram <- function(df, var, binwidth) {
  label <- rlang::englue("A histogram of {{var}} with binwidth {binwidth}")
  df |>
    ggplot(aes(x = {{ var }})) +
    geom_histogram(binwidth = binwidth) +
    labs(title = label)
}
diamonds |> histogram(carat, 0.1)
```



Naming & style

Names are for humans

R doesn't care what you name things, but smart naming makes our lives easier.

- Function names should be **verbs**; arguments should be **nouns**.
  - exceptions: well-known nouns (`mean()`), or accessors (`coef()`).
- Prefer **clear over short** since autocomplete can type the long name for you.

```
f()                     # too short, meaningless
my_awesome_function()   # not a verb, not descriptive
impute_missing()        # long but clear
collapse_years()        # long but clear
```

Whitespace conventions

- `function(...)` is always followed by `{ }`.
- The body is indented **two extra spaces**.
- Put spaces inside `{ }` so the embracing stands out.

```r
# good
density <- function(color, facets, binwidth = 0.1) {
  diamonds |>
    ggplot(aes(x = carat, y = after_stat(density), color = {{ color }})) +
    geom_freqpoly(binwidth = binwidth) +
    facet_wrap(vars({{ facets }}))
}
```

Skimming the left-hand margin should reveal the structure of your code.

Activity

In-class activity

Open `day20-activity.R` in RStudio and work through the parts in order.

- **Part 1**: write & test **vector functions** (`rescale01`, a mutate helper, a summary helper).
- **Part 2**: write **data frame functions** using embracing `{ }` on `nycflights13`.
- **Part 3 (challenge)**: write a **plot function**, use `:=`, and label it with `rlang::englue()`.

Activity file

Wrap-up & looking ahead

What we covered

- Write a function once you've copy-pasted **more than twice**
- **Vector functions**: same-length-out (mutate) vs single-value-out (summary)
- Always **test on known inputs**; improve in **one place**
- **Data frame functions**: embrace data-masking args with `{ }`
- `pick()` for tidy-selection inside data-masking; `:=` for names
- **Plot functions**: embrace inside `aes()`; label with `rlang::englue()`
- Names are **verbs**, clear beats short

Upcoming

- **Tue Jun 16:** control flow: `if/else`, `for`, `while`
- **Reading before Tue:** *R4DS* §26.1–26.3

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 25 · tidyverse style guide